

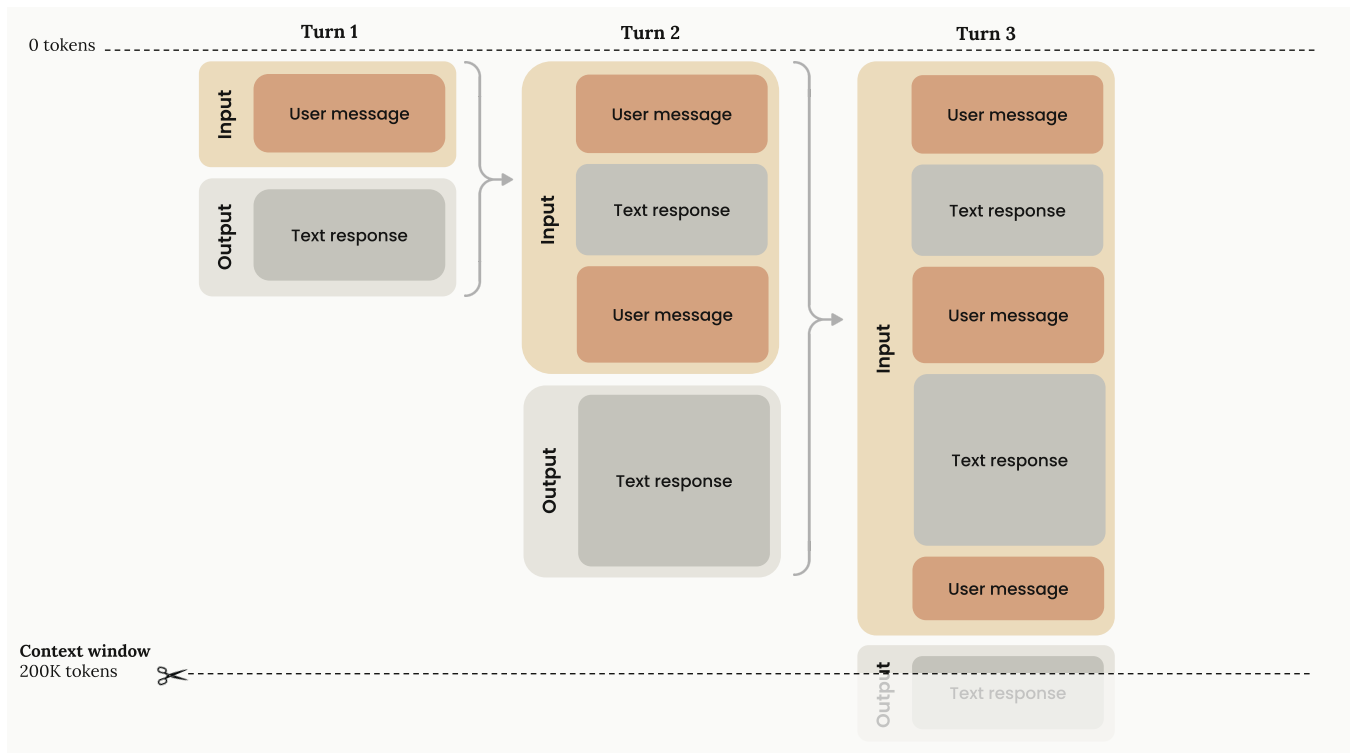
Context windows

[Copy page](#) ▼

Understanding the context window

The "context window" refers to the entirety of the amount of text a language model can look back on and reference when generating new text plus the new text it generates. This is different from the large corpus of data the language model was trained on, and instead represents a "working memory" for the model. A larger context window allows the model to understand and respond to more complex and lengthy prompts, while a smaller context window may limit the model's ability to handle longer prompts or maintain coherence over extended conversations.

The diagram below illustrates the standard context window behavior for API requests¹:



¹For chat interfaces, such as for [claude.ai](#), context windows can also be set up on a rolling "first in, first out" system.



Previous turns are preserved completely.

- **Linear growth pattern:** The context usage grows linearly with each turn, with previous turns preserved completely.
- **200K token capacity:** The total available context window (200,000 tokens) represents the maximum capacity for storing conversation history and generating new output from Claude.
- **Input-output flow:** Each turn consists of:
 - **Input phase:** Contains all previous conversation history plus the current user message
 - **Output phase:** Generates a text response that becomes part of a future input

The context window with extended thinking

When using extended thinking, all input and output tokens, including the tokens used for thinking, count toward the context window limit, with a few nuances in multi-turn situations.

The thinking budget tokens are a subset of your `max_tokens` parameter, are billed as output tokens, and count towards rate limits.

However, previous thinking blocks are automatically stripped from the context window calculation by the Claude API and are not part of the conversation history that the model "sees" for subsequent turns, preserving token capacity for actual conversation content.

The diagram below demonstrates the specialized token management when extended thinking is enabled:

- **Stripping extended thinking:** Extended thinking blocks (shown in dark gray) are generated during each turn's output phase, **but are not carried forward as input tokens for subsequent turns**. You do not need to strip the thinking blocks yourself. The Claude API automatically does this for you if you pass them back.
- **Technical implementation details:**

[Ask Docs](#)



- Extended thinking tokens are billed as output tokens only once, during their generation.
- The effective context window calculation becomes: $\text{context_window} = (\text{input_tokens} - \text{previous_thinking_tokens}) + \text{current_turn_tokens}$.
- Thinking tokens include both `thinking` blocks and `redacted_thinking` blocks.

This architecture is token efficient and allows for extensive reasoning without token waste, as thinking blocks can be substantial in length.

 You can read more about the context window and extended thinking in our [extended thinking guide](#).

The context window with extended thinking and tool use

The diagram below illustrates the context window token management when combining extended thinking with tool use:

1 First turn architecture

- **Input components:** Tools configuration and user message
- **Output components:** Extended thinking + text response + tool use request
- **Token calculation:** All input and output components count toward the context window, and all output components are billed as output tokens.

2 Tool result handling (turn 2)

- **Input components:** Every block in the first turn as well as the `tool_result`. The extended thinking block **must** be returned with the corresponding tool results. This is the only case wherein you **have to** return thinking blocks.
- **Output components:** After tool results have been passed back to Claude, Claude will respond with only text (no additional extended thinking until the



window, and all output components are billed as output tokens.

3 Third Step

- **Input components:** All inputs and the output from the previous turn is carried forward with the exception of the thinking block, which can be dropped now that Claude has completed the entire tool use cycle. The API will automatically strip the thinking block for you if you pass it back, or you can feel free to strip it yourself at this stage. This is also where you would add the next `User` turn.
 - **Output components:** Since there is a new `User` turn outside of the tool use cycle, Claude will generate a new extended thinking block and continue from there.
 - **Token calculation:** Previous thinking tokens are automatically stripped from context window calculations. All other previous blocks still count as part of the token window, and the thinking block in the current `Assistant` turn counts as part of the context window.
- **Considerations for tool use with extended thinking:**
 - When posting tool results, the entire unmodified thinking block that accompanies that specific tool request (including signature/redacted portions) must be included.
 - The effective context window calculation for extended thinking with tool use becomes: `context_window = input_tokens + current_turn_tokens`.
 - The system uses cryptographic signatures to verify thinking block authenticity. Failing to preserve thinking blocks during tool use can break Claude's reasoning continuity. Thus, if you modify thinking blocks, the API will return an error.

① Claude 4 models support [interleaved thinking](#), which enables Claude to think between tool calls and make more sophisticated reasoning after receiving tool results.

Claude Sonnet 3.7 does not support interleaved thinking, so there is no interleaving of extended thinking and tool calls without a non- `tool_result` user turn in between.

For more information about using tools with extended thinking, see our [extended thinking guide](#).

1M token context window

Ask Docs



and work with more extensive codebases.

i The 1M token context window is currently in beta for organizations in [usage tier 4](#) and organizations with custom rate limits. The 1M token context window is only available for Claude Sonnet 4 and Sonnet 4.5.

To use the 1M token context window, include the `context-1m-2025-08-07` [beta header](#) in your API requests:

Python ▾



```
from anthropic import Anthropic

client = Anthropic()

response = client.beta.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "Process this large document..."}
    ],
    betas=["context-1m-2025-08-07"]
)
```

Important considerations:

- **Beta status:** This is a beta feature subject to change. Features and pricing may be modified or removed in future releases.
- **Usage tier requirement:** The 1M token context window is available to organizations in [usage tier 4](#) and organizations with custom rate limits. Lower tier organizations must advance to usage tier 4 to access this feature.
- **Availability:** The 1M token context window is currently available on the Claude API, [Microsoft Foundry](#), [Amazon Bedrock](#), and [Google Cloud's Vertex AI](#).
- **Pricing:** Requests exceeding 200K tokens are automatically charged at premium rates (2x input, 1.5x output pricing). See the [pricing documentation](#) for details.
- **Rate limits:** Long context requests have dedicated rate limits. See the [rate limits documentation](#) for details.
- **Multimodal considerations:** When processing large numbers of images or pdfs, be aware that the files can vary in token usage. When pairing a large prompt with a large

Ask Docs

Context awareness in Claude Sonnet 4.5 and Haiku 4.5

Claude Sonnet 4.5 and Claude Haiku 4.5 feature **context awareness**, enabling these models to track their remaining context window (i.e. "token budget") throughout a conversation. This enables Claude to execute tasks and manage context more effectively by understanding how much space it has to work. Claude is natively trained to use this context precisely to persist in the task until the very end, rather than having to guess how many tokens are remaining. For a model, lacking context awareness is like competing in a cooking show without a clock. Claude 4.5 models change this by explicitly informing the model about its remaining context, so it can take maximum advantage of the available tokens.

How it works:

At the start of a conversation, Claude receives information about its total context window:

```
<budget:token_budget>200000</budget:token_budget>
```



The budget is set to 200K tokens (standard), 500K tokens (Claude.ai Enterprise), or 1M tokens (beta, for eligible organizations).

After each tool call, Claude receives an update on remaining capacity:

```
<system_warning>Token usage: 35000/200000; 165000 remaining</system_warning>
```



This awareness helps Claude determine how much capacity remains for work and enables more effective execution on long-running tasks. Image tokens are included in these budgets.

Benefits:

Context awareness is particularly valuable for:

- Long-running agent sessions that require sustained focus
- Multi-context-window workflows where state transitions matter
- Complex tasks requiring careful token management

[Ask Docs](#)



Context window management with newer Claude models

In newer Claude models (starting with Claude Sonnet 3.7), if the sum of prompt tokens and output tokens exceeds the model's context window, the system will return a validation error rather than silently truncating the context. This change provides more predictable behavior but requires more careful token management.

To plan your token usage and ensure you stay within context window limits, you can use the [token counting API](#) to estimate how many tokens your messages will use before sending them to Claude.

See our [model comparison](#) table for a list of context window sizes by model.

Next steps



Model comparison table

See our model comparison table for a list of context window sizes and input / output token pricing by model.



Extended thinking overview

Learn more about how extended thinking works and how to implement it alongside other features such as tool use and prompt caching.



Ask Docs

Solutions

Learn



Coding

Customer support

Education

Financial services

Government

Life sciences

Partners

Amazon Bedrock

Google Cloud's Vertex AI

Courses

Use cases

Connectors

Customer stories

Engineering at Anthropic

Events

Powered by Claude

Service partners

Startups program

Company

Anthropic

Careers

Economic Futures

Research

News

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

Ask Docs